

Capítulo 13

Elevação de Privilégios Locais

13.1. Objetivos

- Entender a elevação de privilégios
- Procurar por possíveis alvos
- Entender as possibilidades de ataques locais
- Entender a elevação de privilégios
- Procurar por possíveis alvos
- Entender as possibilidades de ataques locais

13.2. O que é escalada de privilégios?

Escalção ou elevação de privilégios basicamente significa adicionar mais direitos ou permissões para um usuário. Em resumo, escalção de privilégios tenta transformar um usuário normal em um usuário administrativo, usuário com maior privilégios do que o usuário atual ou fazer com que um usuário participe de outros grupos locais na máquina, com privilégio diferente do privilégio atual do atacante.

Quando exploramos alguns serviços, nem sempre conseguimos acesso root. Esse é o caso da exploração de um PHP Inject, que vimos anteriormente. Alguns outros exemplos podem ser usados, como exploração de um daemon que não é executado como root. Portanto, para conseguirmos controlar totalmente a máquina e pode executar programas que precisam de privilégios de administrador, precisamos aumentar nosso privilégio localmente.

Porém, a escalção de privilégios não está limitada apenas a aumentar os privilégios dentro do sistema operacional, mas em qualquer sistema. Por exemplo, podemos ter um acesso limitado a um servidor de banco de dados Oracle e desejamos nos tornar DBA, podendo assim acessar todas as tabelas e bases de dados existentes no banco. O ato de tornar um usuário com mais privilégios é também chamado de elevação de privilégios.

Exemplos de caso onde podemos realizar o ataque:

- Exploração de aplicação web que não é executada como root;
- Exploração de serviços que não são executados como root ou tem seu privilégio “dropado”
- Exploração interna de um aplicativo, por exemplo, um Banco de Dados Oracle.
- Quando conseguimos uma senha local, sem privilégio administrativo.
Ex.: Bruteforce em servidor ssh

13.3. Possíveis alvos

Normalmente, aplicações que possuem `suidroot` são as mais exploradas, além do próprio `kernel` do sistema, que é executado com privilégios de super usuário.

Aplicações com `suidroot` é uma aplicação com uma permissão especial conhecida por `suid` bit (“s”), que quando executada, será executada com privilégios do usuário `root`. Portanto, se conseguirmos explorar um arquivo que possui esse tipo de permissão, provavelmente conseguiremos obter os privilégios do usuário `root` no sistema, conseguindo assim controle total.

Para procurarmos por arquivos com `suidroot` na máquina, podemos executar o comando abaixo:



```
# find / -perm -4000 > suidroot.txt  
# less suidroot.txt  
# find / -perm -04000 -exec ls -l {} \;
```

Além de arquivos com `suidroot`, também podem ser exploradas falhas no `kernel` do sistema, que é quem cuida dos privilégios dos usuários, e conseqüentemente, ganharemos privilégios administrativos se a falha for explorada com sucesso.

Portanto, após identificarmos os arquivos que possuem esse tipo especial de permissão, precisamos identificar falhas de segurança nesses softwares. Essas falhas podem ser públicas, ou seja, que é possível encontrar na internet, ou privada, que pode ser comprada de um pesquisador ou empresa, ou até mesmo descoberta pelo próprio atacante.

13.4. Passo a passo

1. Obter acesso local ao sistema
2. Procurar possíveis alvos
3. Tentar explorar esses alvos

4. Acessar as informações já com privilégio maior do que o privilégio anterior

13.5. Laboratório

13.5.1. Desafio 1

1. Através do PHP Shell, existente em uma das máquinas da rede de teste, descobrir qual a versão do kernel da máquina alvo.
2. Buscar um exploit que possa conseguir o shell a partir de alguma vulnerabilidade da versão atual do kernel que está sendo executado.

13.5.2. Desafio 2

1. A outra tarefa, é explorar o arquivo com SUID root que se encontra em uma das máquinas da rede de teste.
2. Crie um trojan local no sistema que permitirá com que o atacante obtenha privilégios de root após executar um interpretador de comando.

13.6. Contramedidas

- Manter o sistema atualizado.
- Deixar com suidroot apenas os arquivos necessários para o funcionamento do sistema.

Capítulo 20

Apagando Rastros

20.1. Objetivos

- Entender a importância de cobrir rastros
- Conhecer as técnicas para encobrir suas ações
- Conhecer as ferramentas empregadas

20.2. Por que encobrir rastros?

Um dos objetivos, em um teste de invasão, de utilizar técnicas para encobrir seus rastros e ações, é para testar a eficácia e competência do time de resposta a incidentes e perícia forense caso os mesmos existam.

As técnicas para apagar rastros também são conhecidas como “anti-forensic”.

Não há necessidade desse tipo de ação por parte de um pentester, caso queira deixar as evidências de exploração para posterior análise por parte da equipe de Ti da empresa contratante. No entanto, caso tenha como objetivo testar, também, a capacidade da equipe de perícia forense em investigar um caso de invasão, é interessante implementar os passos estudados nesse capítulo.

20.3. O que encobrir?

- Logs de IDS

- Onde são armazenadas todas as evidências de tráfego anormal que tenha sido detectado na rede. Isso inclui desde o envio de arquivos maliciosos à varreduras no sistema, em busca de informações.

- Logs de Firewall

- Logs que guardam as informações filtradas por regras de firewall. Normalmente os administradores, quando criam as regras de firewall, tem por hábito mandar armazenar em log tentativas de varreduras, ataques de brute force e acesso sem autorização a serviços específicos.

- Arquivos copiados no sistema

- Qualquer arquivo que tenha sido copiado para o sistema, mesmo que posteriormente seja apagado, deixa rastros que podem ser recuperados com ferramentas específicas.

- Arquivos sendo executados, como backdoors, por exemplo
 - Todo programa ou arquivo em execução, é reconhecido pelo sistema como um processo, e como um pode ser recuperado da memória. Existem várias formas de mascarar a execução de binários, como por exemplo um rootkit, que substitui binários do sistemas por seus próprios, com implementações de códigos maliciosos.
- Logs de comandos
 - Tudo o que é digitado no terminal é armazenado no `.bash_history` do usuário, por exemplo. Mesmo que seja apagado, esse arquivo também pode ser recuperado pela equipe de perícia forense.
- Logs de sessão
 - Quando efetuamos o login e autenticamos uma sessão válida, tudo o que ocorre na mesma é armazenado em logs. Algumas organizações possuem, inclusive, servidores exclusivos para armazenamento e gerenciamento de logs. No Linux, a maioria dos logs ficam armazenados em `/var/log`.

20.4. Técnicas

- Sobreescrita de dados
 - Quando apagamos algo em um disco, os dados são apenas marcados para a deleção e não realmente apagados. Os dados marcados para a deleção, são apagados apenas quando o sistema operacional utiliza os mesmos blocos do disco para gravar novos dados, realizando a sobreescrita. Quanto mais vezes aquele mesmo setor for sobreescrito, mais difícil se tornará a recuperação das informações originalmente existentes. Esse método também é conhecido como “wipe”.
- Prevenção de criação de dados
 - É possível, através da alteração de permissão em determinados arquivos, que novos dados sejam inseridos no mesmo, por exemplo. Podemos citar o caso de arquivos de log, que se tiver sua permissão alterada para negar

a possibilidade de escrita nos mesmo, nenhuma nova operação será armazenada, e o administrador do sistema não poderá fazer a verificação posterior para entender o que comprometeu o sistema ou a rede.

- **Encriptação de dados**

- A melhor maneira de ocultar um arquivo, para que ninguém veja seu conteúdo, ou consiga alterá-lo, é encriptando-o, seja através de uma ferramenta específica de encriptação, seja ocultando o arquivo dentro de outro, cuja extensão e conteúdo sejam diversos do original. Essa última técnica também pode ser chamada de esteganografia.

- **Deleção segura de dados**

- Essa técnica está diretamente vinculada com a primeira, de sobreescrita de dados. Todo e qualquer processo de deleção de arquivos, deve ser cuidadoso, para que não seja possível a posterior recuperação das informações.

20.5. Ferramentas

- **Tor (The Onion Router)**

- O Tor mantém o usuário livre de bisbilhoteiros, inclusive os do FBI e os da CIA, e impede (ou dificulta bastante) qualquer tipo de rastreamento. É exatamente isso que o Tor oferece. Em vez de seguir uma rota direta entre origem e destino, toda a informação transmitida por ele segue um caminho randômico, que se altera permanentemente, através de diversos servidores voluntários que cobrem a rota. Fica difícil para qualquer sistema saber quem você é, onde você está ou de onde veio, embora seja possível saber o que você está levando consigo.

- **Wipe**

- Wipe é um aplicativo que permite a deleção segura de dados, permitindo que o usuário defina quais arquivos serão apagados e quantas vezes aqueles blocos de disco, onde os arquivos apagados estavam alocados,

serão sobreescritos. Quanto mais vezes se sobreescreve, mais difícil a posterior recuperação dos dados. Cada operação de sobreescrita deve ser realizada até o final, para que o programa seja completamente eficaz.

- Scrub

- Outra possibilidade para realizar o “data wiping”, sobrescrevendo os dados deletados com um padrão determinado de informações, que podem ou não ser removidas no final da informação. Se não forem removidas, o perito forense encontrará apenas “lixo digital” nos blocos do disco, sem qualquer coerência.

- Steghide

- Steghide é um programa de esteganografia que é capaz de esconder dados em vários tipos de arquivos de áudio e de imagem. As frequências de som e de cor, respectivamente, não são alteradas tornando o arquivo resistente contra testes estatísticos de primeira ordem. Formatos de arquivos JPEG, BMP, WAV e AU são suportados para uso como arquivo de "cobertura". Não há restrições sobre o formato dos dados secretos. O algoritmo de criptografia padrão é o Rijndael com uma chave de 128 bits de comprimento (que é AES - Advanced Encryption Standard). Se você não confia nesta combinação por qualquer razão, sinta-se à vontade para escolher outra combinação modo/algoritmo.

20.6. Contramedidas

- Instalar Host IDS em todas as máquinas
- Manter as regras de firewall e Network IDS bem configuradas
- Gerenciar logs em servidores próprios e bem protegidos
- Gerenciar permissões em servidores, utilizando ferramentas como o SELinux, por exemplo